

자연들에 — RAG 처리 상세 프로세스 (Top-Down)

Top-Down 상세 프로세스 | 기준: agri_ai_core/ | 작성일: 2026-04-18

R1. 문서 적재 API — 파일 업로드 RAG

#	디렉토리	파일명	함수명	함수기능역할
1	api/	app.py	rag_perform	/api/v1/rag/perform 엔트리. 첨부 파일을 받아 임시 저장 → 문서 처리 파이프라인 호출.
2	api/	app.py	rag_save	/api/v1/rag/save 엔트리. 세션 대화 내용을 RAG 문서로 변환·저장.
3	api/	models.py	RagRequest / RagSaveRequest / RagResponse	파일 경로·farm_id·원본명 등 입력 스키마와 저장 결과 응답 스키마.

R2. 파일 전처리 — 유형별 파싱

#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/	file_processor.py	process_uploaded_files	파일 확장자별 개별 파서 dispatch. 100MB 초과 파일 거부.
2	src/ai/	file_processor.py	read_pdf_file	pypdf 기반 PDF 텍스트 추출 (페이지별 순차 결합).
3	src/ai/	file_processor.py	read_csv_file / read_excel_file	pandas DataFrame 로드 후 마크다운 표로 변환 (최대 100행).
4	src/ai/	file_processor.py	read_text_file	일반 텍스트 파일 읽기 (최대 10000자).
5	src/ai/	file_processor.py	_format_dataframe	DataFrame을 헤더/구분선 포함 마크다운 표로 정형화.

R3. 문서 처리 파이프라인 — 유형 감지·청크 저장·enrichment

#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/rag/	document_processor.py	llm_document_process	RAG 메인 진입. 파일/텍스트 → 유형 감지 → 청크 분할 → VectorDB 저장 → 백그라운드 enrichment 예약.
2	src/ai/rag/	document_processor.py	detect_document_type	파일명/내용 패턴으로 작물 가이드/매뉴얼/보고서/일반 문서 유형을 분류.
3	src/ai/rag/	document_processor.py	_store_crop_chunks_to_farm_knowledg	작물별 메타데이터를 부여해 farm_knowledge 컬렉션에 청크 단위 저장.
4	src/ai/rag/	document_processor.py	_background_enrich	daemon Thread로 document_enricher를 비동기 호출 (API 응답을 블로킹하지 않음).
5	src/ai/rag/	document_processor.py	process_attached_files	LLM 질의에 첨부된 파일을 임시 RAG로 즉시 적재 (세션 범위).
6	src/ai/rag/	document_processor.py	messages_to_text	대화 메시지 리스트를 RAG 문서 형식(단일 텍스트)로 직렬화.

R4. 청킹 (의미 단위 분할)

#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/rag/	chunker.py	chunk_document	1000자 청크 / 200자 오버랩. 문단·문장 경계 우선, 파일명 프리픽스 각 청크에 삽입.
2	src/ai/rag/	chunker.py	store_document_with_chunks	청크 생성 후 각 청크를 임베딩해 upsert_documents_with_embedding으로 일괄 저장.

3	src/ai/rag/	constants.py	(CHUNK_SIZE / CHUNK_OVERLAP / PREFIX_LEN)	청킹 상수 단일 원천. document_processor/document_enricher가 공유.
---	-------------	--------------	---	--

R5. 임베딩 생성 — Ollama bge-m3

#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/	embedder.py	embed_text	단일 텍스트 임베딩. 재시도 5회·동적 타임아웃(텍스트 길이 기반)·health 체크 포함.
2	src/ai/	embedder.py	get_dynamic_timeout	텍스트 길이에 비례해 60~180초 범위 타임아웃 산출.
3	src/ai/	embedder.py	check_ollama_health	Ollama /api/tags 호출로 임베딩 모델 설치·기동 상태 확인.
4	src/ai/	embedder.py	generate_dummy_embedding	임베딩 실패 시 1024차원 더미 벡터 반환 (DB 저장 중단 방지).
5	src/ai/	embedder.py	_extract_embedding_from_payload	Ollama 응답 payload 스키마 변동에 대응한 안전 파서.

R6. VectorDB 저장 — ChromaDB HTTP

#	디렉토리	파일명	함수명	함수기능역할
1	src/chroma/	operations.py	add_document	단일 문서 저장. 임베딩 포함 upsert 호출 (collection.add 대체).
2	src/chroma/	operations.py	upsert_documents_with_embedding	대량 청크 배치 업서트. 임베딩·메타-ID를 JSON-safe로 정제 후 HTTP POST.
3	src/chroma/	operations.py	upsert_collection_data	청크 단일 업서트 (재처리/중복 시 덮어쓰기).
4	src/chroma/	operations.py	set_embed_fn	DI 혹은 상위 계층(embedder)이 기동 시 임베딩 함수를 주입 (역호출 방지).
5	src/chroma/	client.py	_request / get_or_create_collection	ChromaDB v2 REST API 호출. 컬렉션 ID TTL 캐시·heartbeat·자동 생성.
6	src/chroma/	utils.py	_sanitize_metadata / _build_doc_id	JSON-직렬화 가능한 메타만 허용, 결정론적 문서 ID 생성 (중복 업서트 키).
7	src/chroma/	collections.py	farm_knowledge_collection / doc_knowledge_collection	용도별 컬렉션 이름 반환 (farm·doc·conv·memory·web_knowledge).

R7. 문서 Enrichment — LLM 요약·QA 생성

#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/rag/	document_enricher.py	enrich_document	Enrichment 메인. 요약 생성 → QA 쌍 생성 → enriched 청크 저장을 순차 실행. 백그라운드 Thread에서 호출.
2	src/ai/rag/	document_enricher.py	_generate_summary	LLM에게 문서 요약을 지시 (토큰 한도·청크 분할 포함).
3	src/ai/rag/	document_enricher.py	_generate_qa_pairs	문서로부터 Q-A 쌍을 N개 생성해 검색 hit을 상승.
4	src/ai/rag/	document_enricher.py	_store_enriched_data	요약/QA를 별도 메타(type=summary, type=qa)로 farm_knowledge에 추가 저장.
5	src/ai/rag/	document_enricher.py	_parse_qa_response	LLM 응답을 Q/A 블록으로 파싱 (형식 위반 시 버림).
6	src/ai/rag/	document_enricher.py	_call_llm	enrichment 전용 LLM 호출 (타임아웃·num_predict 별도 설정).

R8. 질의 시 RAG 검색 (Retrieval) — 벡터 유사도				
#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/pipeline/	data_collector.py	DataCollector.collect	Stage 2에서 search_farm_knowledge 도구 호출을 예약·실행.
2	src/ai/	tools_data.py	search_farm_knowledge	RAG 검색 진입점. 질문 임베딩 → where 조건 구성 → ChromaDB 질의 → reranker 재정렬.
3	src/ai/	tools_data.py	_build_chroma_where_conditions	farm_id·auth_farm_id·house_id·doc_type 기반 권한/필터 where 절 구성.
4	src/ai/	embedder.py	embed_text	사용자 질의를 1024차원 벡터로 변환 (검색 쿼리용).
5	src/chroma/	operations.py	query_documents	query_embeddings·n_results·where·include로 벡터 유사도 검색 수행.
6	src/ai/	reranker.py	rerank_results	검색 Top-K 후보를 LLM에게 질문·문서 관련성 점수화 지시 → 재정렬. 실패 시 거리 기반 폴백.
7	src/ai/	reranker.py	_build_rerank_prompt / _parse_scores	재정렬용 프롬프트 구성·LLM 점수 응답 파서.

R9. 대화 VectorDB — 멀티턴 하이브리드 메모리				
#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/	conversation_context.py	load_hybrid_context	질의마다 호출. 최근 턴(DB) + 과거 유사 턴(Vector) 병합해 LLM 컨텍스트 구성.
2	src/ai/	conversation_context.py	_search_related_conversations	질문을 임베딩 후 conversation_memory 컬렉션에서 Top-K 유사 턴 회수.
3	src/ai/	conversation_context.py	_async_vectordb_save	응답 완료 직후 daemon Thread로 (질문+응답) 임베딩·업서트.
4	src/ai/	conversation_context.py	_prune_old_conversations	farm_id당 30건 초과 시 오래된 턴부터 삭제 (컬렉션 팅창 방지).

R10. 생육 기반 인과 RAG — 주기 저장				
#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/learning/	growth_rag_processor.py	run_growth_rag	APScheduler에 등록(매일 12:00 / 00:05). 생육 신규 입력 기준으로 환경 통계 RAG 문서 생성.
2	src/ai/learning/	growth_rag_processor.py	_ensure_daily_rag	일자별 누락 보장 (자정 호출 시 is_midnight=True). 미처리 재배포사만 실행.
3	src/ai/learning/	growth_rag_processor.py	_build_rag_document	센서/릴레이/생육 통계를 사람이 읽기 쉬운 RAG 문서 형식으로 조립.
4	src/ai/learning/	growth_rag_processor.py	_build_rag_metadata	farm_id·house_id·crop·period·doc_type 등 필터용 메타를 구성.
5	src/ai/learning/	growth_rag_processor.py	_store_growth_rag	완성된 문서를 farm_knowledge 컬렉션에 청크 단위 저장 (store_document_with_chunks 경유).
6	src/ai/learning/	growth_rag_processor.py	_get_sensor_stats / _get_day_night_stats / _get_moving_averages	PostgreSQL 집계(주야/이동평균/트렌드)로 RAG 본문에 들어갈 통계 산출.

R11. 웹 검색 결과 RAG 캐싱 — 자동 학습				
#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/	tools_search.py	search_web	웹 검색 완료 후 결과를 web_knowledge 컬렉션에 자동 캐싱.
2	src/ai/	tools_search.py	_cache_web_results_to_vectordb	daemon Thread로 검색 결과(제목+요약)를 임베딩·업서트. 재질의 시 재사용.

3	src/ai/	tools_search.py	_filter_relevant_results / _build_retry_query	관련성 필터(위치/키워드)·실패 시 재검색 쿼리 재구성.
4	src/ai/	web_search_engines.py	_search_via_api	Naver + SearXNG + Brave 병렬 호출 라우터 (한국어·영어 분기).
5	src/ai/	web_url_fetcher.py	fetch_url_content	검색 결과 URL 본문을 HTTP fetch + HTML 정제. auto_fetch_urls가 상위 3건 병렬 수집.

R12. 청크 정리 — 주기 유지보수

#	디렉토리	파일명	함수명	함수기능역할
1	src/control/	task_scheduler.py	_chunk_cleanup_job	매일 03:00 실행. farm_knowledge에서 record_datetime 기준 180일 이전 청크를 100개 배치로 삭제.
2	src/chroma/	operations.py	delete_document	ID 리스트 기반 대량 삭제 (collection.delete HTTP 호출).
3	src/chroma/	operations.py	get_documents	메타만 include하여 전체 문서 스캔 → 삭제 대상 ID 추출.

R13. ChromaDB 연결 기반구조

#	디렉토리	파일명	함수명	함수기능역할
1	src/chroma/	client.py	heartbeat	기동 시 연결 확인. 실패 시 startup이 3회 재시도 후 MCP fallback 경로 활성화.
2	src/chroma/	client.py	get_or_create_collection	컬렉션 미존재 시 자동 생성 + ID를 TTL 캐시에 등록.
3	src/chroma/	config.py	CHROMA_HOST / CHROMA_PORT / COLLECTIONS	ChromaDB 연결 설정 및 컬렉션 이름 상수 (port=8000).
4	src/ai/	mcp_client.py	call_tool (chromadb)	direct-HTTP 실패 시 MCP chromadb 서버 경유 호출로 폴백 (대안 경로).